Smart Drone Hangar

Anno Accademico 2025-2026

Autori:
Davide Mancini
Filippo Ricciotti
Martina Malagoli

17 ottobre 2026

Indice

1 Introduzione **2**
1.1 Obiettivo 2
1.2 Componenti utilizzati 3

2 Architettura **4**
2.1 Task implementate 4

3 Progettazione schemi **5**

4 Progettazione circuito **8**
4.1 Schema 8

5 Applicazione Java **9**
5.1 Interfaccia grafica 9
5.2 Comunicazione Arduino-Applicazione 10
5.2.1 Messaggi Arduino → DRU 10
5.2.2 Messaggi DRU → Arduino 11
5.2.3 Implementazione 11

Capitolo 1

Introduzione

1.1 Obiettivo

L'obiettivo del progetto è creare un hangar intelligente per un drone. Il sistema è stato considerato nei suoi due sottosistemi:

- **Drone Hangar (Arduino)**: gestisce l'apertura e chiusura dell'hangar, controlla la temperatura e segnala lo stato di allarme quando raggiunge una certa temperatura. L'Arduino manda delle informazioni sullo stato corrente dell'hangar all'applicazione Java tramite seriale.
- **Drone Remote Unit (PC)**: applicazione Java che simula il comportamento del drone. La DRU manda dei comandi tramite seriale all'hangar quando si vuol fare uscire o entrare e permette la lettura di dati come la distanza del drone e lo stato dell'hangar. Utilizza Swing come interfaccia grafica.

1.2 Componenti utilizzati

Componente	**Modello**	**Pin**	**Interfaccia**
Sensore di presenza	PIR	2	Input
Sensore di prossimità sonar ECHO	HC-SR04	3	Input
Sensore di prossimità sonar TRIG	HC-SR04	4	Output
Bottone	Standard	6	Input
LED verde 1	Standard	7	Output
LED verde 2	Standard	8	Output
LED rosso	Standard	9	Output
Servo motore	SG90	11	PWM
Thermistor	NTC	A0	Analog
I2C LCD	Standard	A4/A5	Output

Capitolo 2

Architettura

L'architettura scelta per l'implementazione è task-based. Sono state quindi realizzate una serie di task, sotto riportate, che vengono gestite da uno scheduler cooperativo.

2.1 Task implementate

- **AlarmTask**: gestisce lo stato dell'allarme in base alla temperatura corrente. Quando l'hangar è in allarme si può resettare solo se la temperatura è tornata sotto la soglia di preallarme.
- **BlinkingTask**: gestisce il blinking del secondo led durante LANDING e TAKEOFF.
- **HangarTask**: gestisce la macchina a stati dell'hangar divisa in DRONE INSIDE, DRONE TAKEOFF, DRONE OUTSIDE, DRONE LANDING.
- **HangarDoorTask**: gestisce l'apertura e chiusura della porta dell'hangar in base allo stato di quest'ultimo. Quando l'hangar è in DRONE TAKEOFF o DRONE LANDING apre la porta e informa tramite il Context condiviso le altre task del completamento dell'apertura (poiché non è un'azione immediata). Quando l'hangar è in DRONE INSIDE o DRONE OUTSIDE e la porta è aperta, inizia a chiuderla.
- **LCDTask**: mostra lo stato corrente dell'hangar all'operatore.
- **SerialCommTask**: gestisce lo scambio di comandi tra l'Arduino e l'applicazione Java.

Capitolo 3

Progettazione schemi

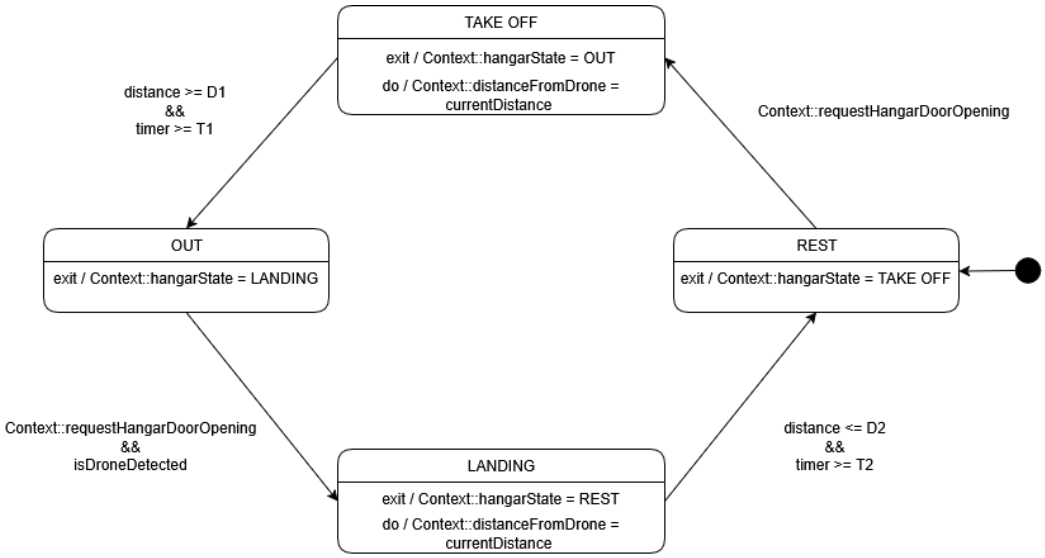


Figura 3.1: Macchina a stati dell'Hangar

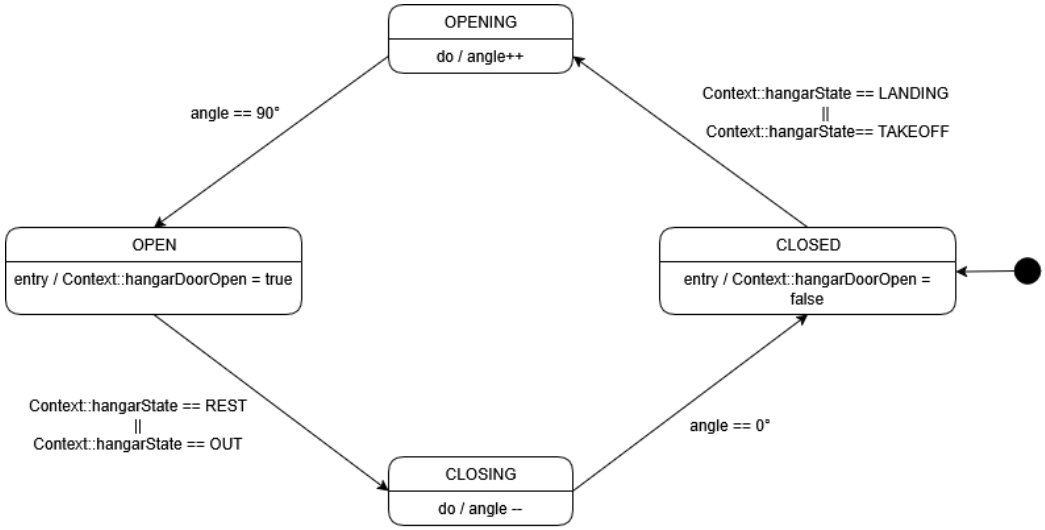


Figura 3.2: Macchina a stati della porta dell'Hangar

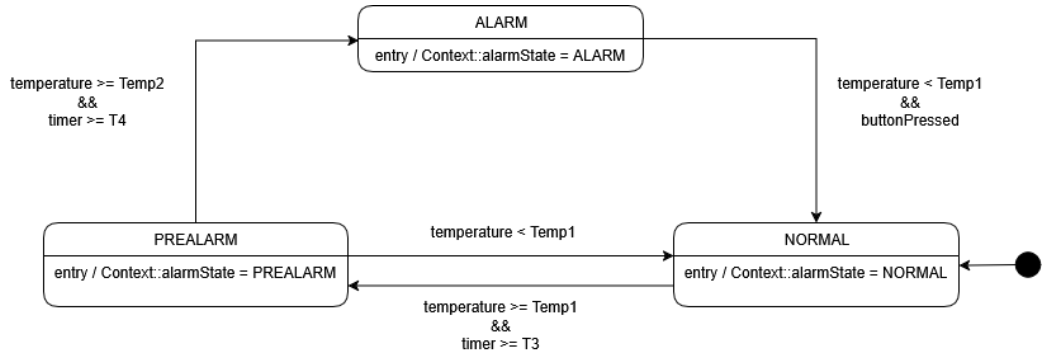


Figura 3.3: Macchina a stati dell'allarme

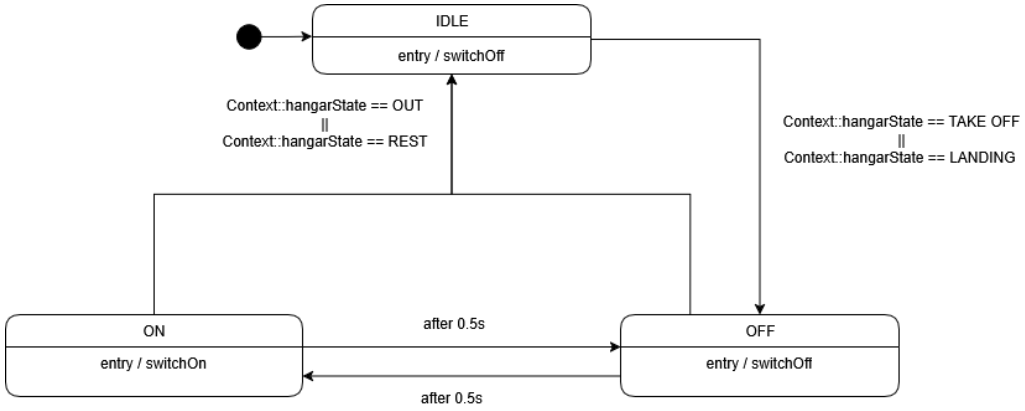


Figura 3.4: Macchina a stati del blinking del led

Capitolo 4

Progettazione circuito

4.1 Schema

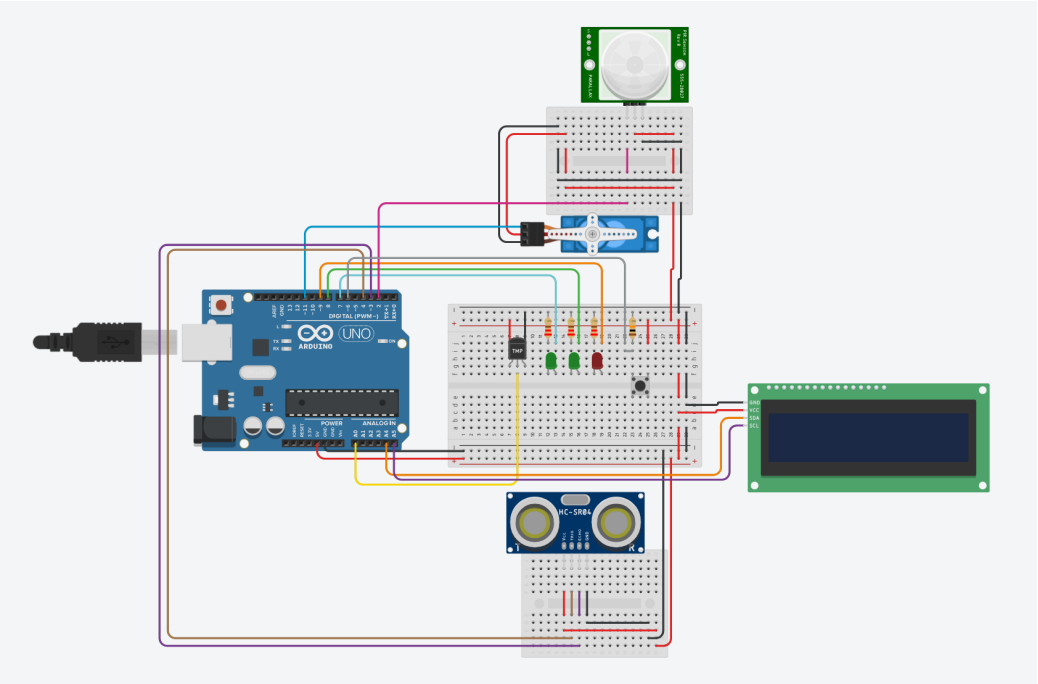


Figura 4.1: Schema Tinkercad rappresentante il circuito

Rispetto all'implementazione fisica, nel corrente schema si è scelto di inserire il termoresistore TMP36 poiché su Tinkercad non è presente il termistore NTC utilizzato nel progetto.

Capitolo 5

Applicazione Java

5.1 Interfaccia grafica

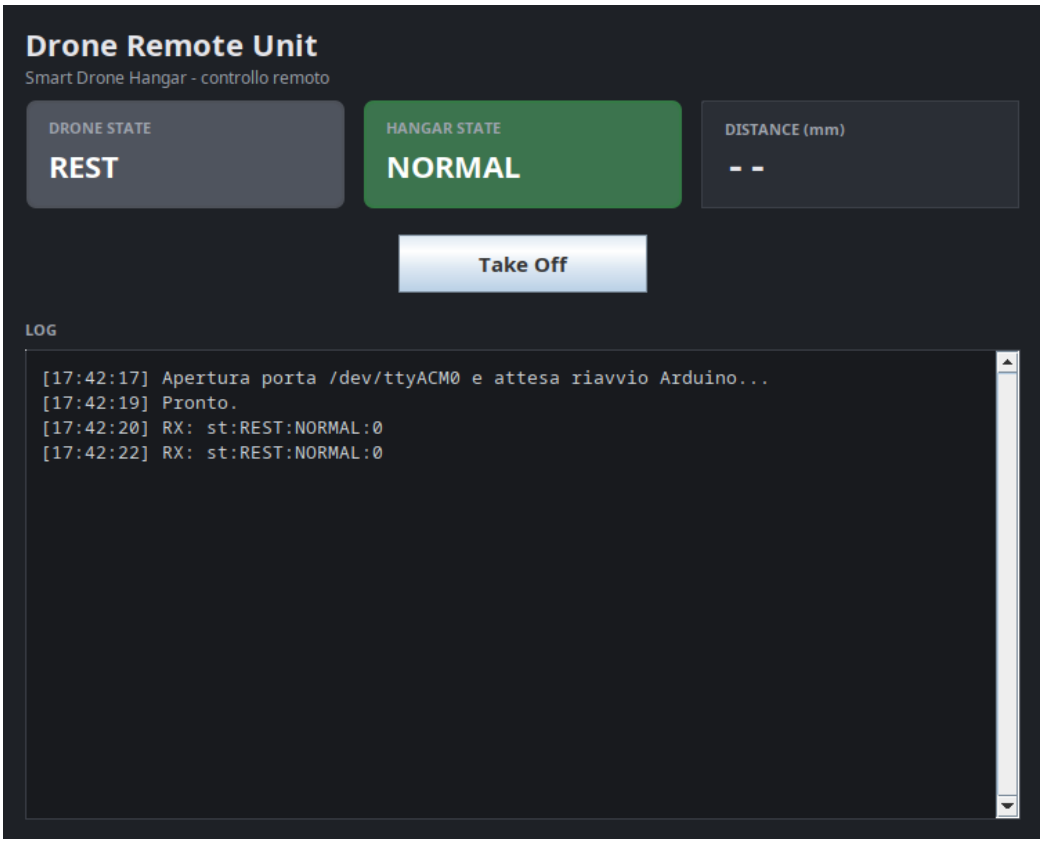


Figura 5.1: Applicazione Drone Remote Unit

5.2 Comunicazione Arduino-Applicazione

La comunicazione tra l'Arduino e la DRU avviene su una singola linea seriale a **115200 baud**, con un protocollo testuale a righe: ogni messaggio è una stringa che termina con un carattere di nuova riga (`\n`), composta da un prefisso a due lettere seguito da `:` e dal payload. La definizione dei prefissi e delle costanti è condivisa tra `Protocol.h` lato Arduino e `Protocol.java` lato DRU, in modo che le due parti non possano disallinearsi.

5.2.1 Messaggi Arduino → DRU

Prefisso	Formato	Descrizione
`st:`	`st:<droneState>:<hangarState>:<distanceCm>`	Stato corrente, inviato periodicamente ad ogni tick di `SerialCommTask`.
`al:`	`al:ALARM`	Notifica di allarme, inviata una tantum quando l'hangar entra in stato di allarme con il drone fuori dall'hangar.
`lo:`	`lo:<messaggio>`	Messaggio diagnostico/di log, mostrato nel pannello di log della DRU (es. rilevamento del drone da parte del PIR).

Tabella 5.1: Messaggi inviati dall'Arduino alla DRU

I campi di `st:` possono assumere i seguenti valori:

- **droneState**: `REST`, `TAKEOFF`, `OUT`, `LANDING`;
- **hangarState**: `NORMAL`, `PREALARM`, `ALARM`;
- **distanceCm**: distanza misurata dal sonar, in centimetri; vale `-1` quando non è disponibile una lettura valida (drone non rilevato, oppure fuori dalle fasi di decollo o atterraggio).

5.2.2 Messaggi DRU → Arduino

Prefisso	Formato	Descrizione
`cmd:`	`cmd:OPEN`	Richiesta di apertura della porta dell'hangar, inviata dalla DRU quando l'operatore simula un comando di decollo o di atterraggio del drone.

Tabella 5.2: Messaggi inviati dalla DRU all'Arduino

Il comando `cmd:OPEN` è unico per entrambe le fasi: non è la DRU a specificare se si tratta di un decollo o di un atterraggio, ma è l'Arduino stesso, tramite `HangarTask`, a interpretarlo in base allo stato corrente dell'hangar (drone dentro → richiesta di decollo, drone fuori → richiesta di atterraggio).

5.2.3 Implementazione

Lato Arduino la comunicazione è gestita da **SerialCommTask**: ad ogni tick legge in modo non bloccante i byte disponibili sulla seriale, riconosce i comandi completi (terminati da `\n`) e li traduce in richieste sul `Context` condiviso, oltre a inviare periodicamente lo stato corrente e, quando necessario, la notifica di allarme. Lato DRU, **SerialCommChannel** incapsula l'accesso alla porta seriale (tramite la libreria JSSC) esponendo un'interfaccia a messaggi asincrona, mentre **MonitoringAgent** è il thread che consuma i messaggi in arrivo, li instrada in base al prefisso e aggiorna di conseguenza l'interfaccia grafica (`DroneRemoteUnitView`).